

Documentation Writer — Screenshot & Build Guide (Reference)

File này chứa hướng dẫn kỹ thuật chi tiết về build ứng dụng và chụp screenshot. Được tách từ `documentation-writer.md` theo nguyên tắc Progressive Disclosure — chỉ đọc khi cần, không tải vào context mặc định.

>

Đọc file này khi: bắt đầu Bước 0.5 (chạy ứng dụng thật) hoặc Bước 2 (chụp màn hình).

A. Build Windows Forms (Release)

```
# Phương án 1: Dùng build.ps1 nếu có sẵn trong project
.\build.ps1

# Phương án 2: dotnet publish (tạo thư mục publish sạch)
dotnet publish -c Release -o ./publish

# Phương án 3: MSBuild trực tiếp
msbuild /p:Configuration=Release /p:OutputPath=./bin/Release/

# Xác nhận build thành công
Get-ChildItem -Path ".\publish", ".\bin/Release" -Filter "*.exe" -Recurse |
    Select-Object Name, LastWriteTime, @{N='SizeMB';E={ [Math]::Round($_.Length/1MB,2) }}
```

Quy tắc: Phải chạy đến khi xuất hiện thông báo **Build succeeded**. Nếu lỗi → DỪNG, báo Senior Developer. Chạy `.exe` Release để xác nhận khởi động bình thường trước khi chụp.

```
# Khởi động ứng dụng Release
$app = Start-Process ".\publish\[AppName].exe" -PassThru
Start-Sleep -Seconds 4
if ($app.HasExited) { Write-Error "ỨNG DỤNG KHÔNG KHỞI ĐỘNG ĐƯỢC — DỪNG LẠI" }
else { Write-Host "OK Ứng dụng đang chạy — PID: $($app.Id)" }
```

B. Khởi động ứng dụng Web

```
# Khởi động server local
npm run dev          # hoặc: python manage.py runserver / dotnet run / ...

# Xác nhận đang chạy — phải trả về HTTP 200
curl -s -o /dev/null -w "%{http_code}" http://localhost:3000/

# Kết quả phải là: 200
```

C. Chụp màn hình — Windows Forms (PowerShell)

```
Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing

function Capture-Window {
    param([string]$ProcessName, [string]$OutputPath)
    $proc = Get-Process -Name $ProcessName -ErrorAction SilentlyContinue | Select-
```

```
Object -First 1
    if (-not $proc) { Write-Host "Không tìm thấy process: $ProcessName"; return }

    $hwnd = $proc.MainWindowHandle
    Add-Type @"
using System;
using System.Drawing;
using System.Runtime.InteropServices;
public class WinCapture {
    [DllImport("user32.dll")] public static extern bool GetWindowRect(IntPtr h, out
RECT r);
    [DllImport("user32.dll")] public static extern bool SetForegroundWindow(IntPtr h);
    public struct RECT { public int Left, Top, Right, Bottom; }
    public static Bitmap Capture(IntPtr hwnd) {
        SetForegroundWindow(hwnd);
        RECT r; GetWindowRect(hwnd, out r);
        var bmp = new Bitmap(r.Right-r.Left, r.Bottom-r.Top);
        using (var g = Graphics.FromImage(bmp))
            g.CopyFromScreen(r.Left, r.Top, 0, 0, bmp.Size);
        return bmp;
    }
}
"@

    $bmp = [WinCapture]::Capture($hwnd)
    $bmp.Save($OutputPath, [System.Drawing.Imaging.ImageFormat]::Png)
    $bmp.Dispose()
    Write-Host "Đã chụp: $OutputPath"
}

# Ví dụ sử dụng
$screenshotDir = "docs\user-manuals\screenshots"
New-Item -ItemType Directory -Force -Path $screenshotDir | Out-Null
$app = Start-Process ".\publish\[AppName].exe" -PassThru
Start-Sleep -Seconds 3
Capture-Window -ProcessName "[AppName]" -OutputPath "$screenshotDir\main-default.png"
```

D. Chụp màn hình — Web (Playwright)

```
pip install playwright && playwright install chromium

from playwright.sync_api import sync_playwright
import os

SCREENS = [
    # (route, tên_file, mô_tả)
    ("/", "home-default", "Trang chủ"),
    ("/login", "login-default", "Đăng nhập - mặc định"),
    ("/login", "login-error", "Đăng nhập - lỗi"),
    ("/dashboard", "dashboard-default", "Dashboard"),
    # ... thêm TẤT CẢ route ở đây
]

BASE_URL = "http://localhost:3000"
OUT_DIR = "docs/user-manuals/screenshots"
os.makedirs(OUT_DIR, exist_ok=True)
```

```
with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page(viewport={"width": 1366, "height": 768})
    for route, fname, desc in SCREENS:
        page.goto(f"{BASE_URL}{route}")
        page.wait_for_load_state("networkidle")
        path = f"{OUT_DIR}/{fname}.png"
        page.screenshot(path=path, full_page=True)
        print(f"OK {desc:30s} -> {path}")
    browser.close()
print(f"\nHoàn thành: {len(SCREENS)} screenshots")
```

E. Tiền điều kiện chạy script chuyển đổi DOCX (Windows)

```
# 1. Đặt encoding UTF-8 (BẮT BUỘC trên Windows – thiếu sẽ crash khi in tiếng Việt)
$env:PYTHONIOENCODING = "utf-8"

# 2. Kiểm tra file output KHÔNG bị mở trong Word
$outputFile = "docs/user-manuals/MANUAL-ten-file.docx"
try {
    $s = [System.IO.File]::Open($outputFile, 'Open', 'ReadWrite', 'None')
    $s.Close()
    Write-Host "OK – file không bị lock"
} catch {
    Write-Error "FILE ĐANG BỊ KHÓA – Đóng file trong Word trước"
}
```

Lỗi thường gặp trên Windows:

- `UnicodeEncodeError: 'charmap' → Thiếu $env:PYTHONIOENCODING = "utf-8"`
- `[Errno 13] Permission denied` → File DOCX đang mở trong Word → Đóng lại
- `(-2147023170, ...)` từ docx2pdf → kill WINWORD.EXE rồi thử lại

F. Quét màn hình theo loại ứng dụng

```
# Windows Forms – liệt kê tất cả Form
Get-ChildItem -Recurse -Filter "*.cs" |
    Select-String -Pattern "class \w+ *: *(Form|UserControl|MetroForm|BaseForm)" |
    ForEach-Object { "$($_.Filename):$($_.LineNumber) – $($_.Line.Trim())" }

# Next.js / React Router
Get-ChildItem -Recurse -Include "*.tsx", "*.jsx" -Path
"src/pages", "src/app", "src/routes"

# ASP.NET MVC
Get-ChildItem -Recurse -Filter "*.cshtml" | Select-Object FullName
```

G. Xuất PDF (các phương án)

```
# Phương án 1: docx2pdf (Windows/macOS – cần MS Word)
pip install docx2pdf
```

```
$env:PYTHONIOENCODING = "utf-8"
python scripts/md_to_docx_kztek.py docs/user-manuals/MANUAL-[feature-slug].md

# Phương án 2: LibreOffice (Linux)
soffice --headless --convert-to pdf docs/user-manuals/MANUAL-[feature-slug].docx --
outdir docs/user-manuals/

# Phương án 3: pypandoc
pip install pypandoc
python -c "import pypandoc; pypandoc.convert_file('docs/user-manuals/MANUAL-[feature-
slug].md', 'pdf', outputfile='docs/user-manuals/MANUAL-[feature-slug].pdf',
extra_args=['--pdf-engine=xelatex'])"
```